Stack queue

23K. Stack & Queue (10 problems)

Pattern recognition: LIFO/FIFO processing, monotonic stack, matching pairs, expression evaluation.

23.141 Valid Parentheses

JAVASCRIPT Copy

```javascript
function isValid(s) {
  const stack = [];
  const pairs = { ')': '(', ']': '[', '}': '{' };
  for (const char of s) {
    if ('({['.includes(char)) stack.push(char);
    else if (stack.pop() !== pairs[char]) return false;
  }
  return stack.length === 0;
}
```

23.142 Min Stack (O(1) getMin)

JAVASCRIPT Copy

```
  push(val) {
    this.stack.push(val);
    this.minStack.push(Math.min(val, this.minStack[this.minStack.length -
  }
  pop() { this.stack.pop(); this.minStack.pop(); }
  top() { return this.stack[this.stack.length - 1]; }
  getMin() { return this.minStack[this.minStack.length - 1]; }
}
```

23.143 Next Greater Element (Monotonic Stack)

JAVASCRIPT Copy

```
function nextGreaterElements(nums) {
  const result = Array(nums.length).fill(-1);
  const stack = [];
  for (let i = nums.length - 1; i ≥ 0; i--) {
    while (stack.length && stack[stack.length - 1] ≤ nums[i]) stack.pop()
    if (stack.length) result[i] = stack[stack.length - 1];
    stack.push(nums[i]);
  }
  return result;
}
```

23.144 Daily Temperatures

Approach: Keep a monotonic decreasing stack of unresolved day indices, and when a warmer day arrives pop each colder day and record the index gap as its answer.

JAVASCRIPT Copy

```
  const stack = []; // indices of days awaiting a warmer temperature
  for (let i = 0; i < temperatures.length; i++) {
    while (stack.length && temperatures[i] > temperatures[stack[stack.leng
      const prev = stack.pop();
      answer[prev] = i - prev;
    }
    stack.push(i);
  }
  return answer;
}
```

Monotonic stack tracking indices, compute differences. Same pattern as Next Greater Element.

23.145 Evaluate Reverse Polish Notation

Approach: Push numbers onto a stack; on each operator pop the two operands, apply it (truncating division toward zero), and push the result back.

JAVASCRIPT Copy

```
function evalRPN(tokens) {
  const stack = [];
  const ops = {
    '+': (a, b) => a + b,
    '-': (a, b) => a - b,
    '*': (a, b) => a * b,
    '/': (a, b) => Math.trunc(a / b),
  };
  for (const token of tokens) {
    if (token in ops) {
      const b = stack.pop();
      const a = stack.pop();
      stack.push(ops[token](a, b));
    } else {
      stack.push(Number(token));
```

```
}
```

Stack-based: push numbers, on operator pop two operands, compute, push result.

23.146 Largest Rectangle in Histogram

Approach: Maintain a stack of indices with increasing heights; when a shorter bar appears, pop taller bars and compute the area each spans using the new left/right boundaries, with a sentinel 0 to flush at the end.

JAVASCRIPT Copy

```
function largestRectangleArea(heights) {
  const stack = []; // indices of increasing heights
  let maxArea = 0;
  for (let i = 0; i ≤ heights.length; i++) {
    const h = i ═══ heights.length ? 0 : heights[i];
    while (stack.length && heights[stack[stack.length - 1]] ≥ h) {
      const height = heights[stack.pop()];
      const width = stack.length ? i - stack[stack.length - 1] - 1 : i;
      maxArea = Math.max(maxArea, height * width);
    }
    stack.push(i);
  }
  return maxArea;
}
```

Monotonic stack — for each bar, find first smaller on left and right. Area = height × (rightBound - leftBound - 1).

23.147 Implement Queue Using Stacks

stack is empty, giving amortized O(1) operations.

JAVASCRIPT Copy

```
class MyQueue {
  constructor() {
    this.inStack = [];
    this.outStack = [];
  }
  push(x) {
    this.inStack.push(x);
  }
  _transfer() {
    if (this.outStack.length === 0) {
      while (this.inStack.length) this.outStack.push(this.inStack.pop());
    }
  }
  pop() {
    this._transfer();
    return this.outStack.pop();
  }
  peek() {
    this._transfer();
    return this.outStack[this.outStack.length - 1];
  }
  empty() {
    return this.inStack.length === 0 && this.outStack.length === 0;
  }
}
```

Two stacks: push to stack1, on dequeue move all from stack1 to stack2 (reverses order), pop from stack2. Amortized O(1).

23.148 Implement Stack Using Queue

Approach: Use a single queue and, after each push, rotate the queue by dequeuing and re-enqueuing every earlier element so the newest value sits at the front for O(1) pop and

JAVASCRIPT Copy

```
class MyStack {
  constructor() {
    this.queue = [];
  }
  push(x) {
    this.queue.push(x);
    // rotate so the newest element moves to the front
    for (let i = 0; i < this.queue.length - 1; i++) {
      this.queue.push(this.queue.shift());
    }
  }
  pop() {
    return this.queue.shift();
  }
  top() {
    return this.queue[0];
  }
  empty() {
    return this.queue.length === 0;
  }
}
```

One queue: on push, enqueue element then dequeue and re-enqueue all previous elements.

23.149 Asteroid Collision

Approach: Iterate left to right using a stack; a left-moving asteroid collides with positive asteroids on top, popping smaller ones and destroying itself (or both) on equal size, and is pushed only if it survives.

JAVASCRIPT Copy

```
function asteroidCollision(asteroids) {
  const stack = [];
```

```
    const top = stack[stack.length - 1];
    if (top < -a) {
      stack.pop(); // top explodes, keep checking further left
    } else if (top === -a) {
      stack.pop(); // both explode
      alive = false;
    } else {
      alive = false; // current asteroid explodes
    }
  }
  if (alive) stack.push(a);
}
return stack;
}
```

Stack: positive goes right, negative goes left. When opposing, larger survives. Equal = both destroy.

23.150 Simplify Path

Approach: Split the path on slashes and process each segment with a stack: ignore empty and '.' segments, pop on '..', and push real directory names, then join the stack back with slashes.

JAVASCRIPT Copy

```
function simplifyPath(path) {
  const stack = [];
  for (const part of path.split('/')) {
    if (part === '' || part === '.') continue;
    if (part === '..') stack.pop();
    else stack.push(part);
  }
  return '/' + stack.join('/');
}
```

QUICK CHECK — no marks, just for you.

1. The lesson's Valid Parentheses solution is run on three inputs. What is printed?

JAVASCRIPT Copy

```
function isValid(s) {
  const stack = [];
  const pairs = { ')': '(', ']': '[', '}': '{' };
  for (const char of s) {
    if ('({['.includes(char)) stack.push(char);
    else if (stack.pop() !== pairs[char]) return false;
  }
  return stack.length === 0;
}

console.log(isValid('()[]{}'));
console.log(isValid('('));
console.log(isValid('(('));
```

- ○ true
false
false
- ○ true
false
true
- ○ true
true
false
- ○ false
false
false

Check answer

JAVASCRIPT Copy

```
function nextGreaterElements(nums) {
  const result = Array(nums.length).fill(-1);
  const stack = [];
  for (let i = nums.length - 1; i >= 0; i--) {
    while (stack.length && stack[stack.length - 1] <= nums[i]) stack.pop();
    if (stack.length) result[i] = stack[stack.length - 1];
    stack.push(nums[i]);
  }
  return result;
}

console.log(JSON.stringify(nextGreaterElements([2, 1, 2, 4, 3])));
```

- ◯ `[4,2,4,-1,3]`
- ◯ `[2,2,4,-1,-1]`
- ◯ `[4,4,4,-1,-1]`
- ◯ `[4,2,4,-1,-1]`

Check answer

3. In the lesson's 'Implement Queue Using Stacks' approach, what does moving all elements from stack1 into stack2 accomplish?

- ◯ It reverses the element order, so the oldest element ends up on top of stack2 and a plain pop gives FIFO behaviour
- ◯ It sorts the elements so the smallest value is dequeued first
- ◯ It keeps stack1 small enough that push can stay O(1) instead of O(n)
- ◯ It makes every dequeue cost O(n), which is the unavoidable price of building a queue from stacks

Previous
Graphs

Next
Strings

LEARN MORE

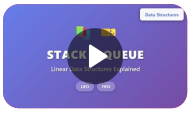
ARTICLES & DOCS

Stack data structure (GfG)
geeksforgeeks.org

VIDEOS



Intro to stacks & queues
YouTube



Stack & queue: LIFO vs FIFO
YouTube



Introduction to stack & queue
YouTube

Hand-picked references for this lesson.